

**RANCANG BANGUN GAME RPG *TURN-BASED AETHERIA  
CHRONICLES* BERBASIS WEB DENGAN FITUR GACHA  
DAN AI ADAPTIF MENGGUNAKAN *BEHAVIOR TREES***

**PROPOSAL PENELITIAN**



**Oleh**  
**ABEL TRIVIA BIMA PRAWIRA**  
**NIM 22051204104**

**UNIVERSITAS NEGERI SURABAYA**  
**FAKULTAS TEKNIK**  
**PROGRAM STUDI S1 TEKNIK INFORMATIKA**  
**TAHUN 2025**

## HALAMAN PERSETUJUAN

NIM : 22051204104  
Nama : Abel Trivia Bima Prawira  
Judul Skripsi : Rancang Bangun Game RPG *Turn-based Aetheria Chronicles* Berbasis Web dengan Fitur Gacha dan AI Adaptif Menggunakan *Behavior Trees*.

Dengan ini telah disetujui dan dinyatakan memenuhi syarat untuk diajukan dalam ujian tugas akhir.

Surabaya, 29 Oktober 2025

**Pembimbing,**

**Anita Qoiriah, S.Kom., M.Kom.**  
**NIP. 196901251995122001**

## DAFTAR ISI

|                                                  |            |
|--------------------------------------------------|------------|
| <b>HALAMAN JUDUL.....</b>                        | <b>i</b>   |
| <b>HALAMAN PERSETUJUAN .....</b>                 | <b>ii</b>  |
| <b>DAFTAR ISI .....</b>                          | <b>iii</b> |
| <b>DAFTAR TABEL.....</b>                         | <b>v</b>   |
| <b>DAFTAR GAMBAR .....</b>                       | <b>vi</b>  |
| <b>BAB I PENDAHULUAN.....</b>                    | <b>1</b>   |
| 1.1    Latar Belakang .....                      | 1          |
| 1.2    Rumusan Masalah .....                     | 4          |
| 1.3    Tujuan Penelitian.....                    | 4          |
| 1.4    Manfaat Penelitian.....                   | 5          |
| 1.5    Batasan Penelitian.....                   | 5          |
| <b>BAB II KAJIAN PUSTAKA.....</b>                | <b>7</b>   |
| 2.1    Kajian Teori .....                        | 7          |
| 2.1.1    Konsep Dasar Pengembangan.....          | 7          |
| 2.1.2    Mekanisme Pertarungan .....             | 12         |
| 2.1.3    Kecerdasan Buatan (AI) dalam Game ..... | 17         |
| 2.1.4    Arsitektur Sistem Berbasis Web.....     | 18         |
| 2.2    Penelitian Terdahulu.....                 | 23         |
| <b>BAB III METODE PENELITIAN.....</b>            | <b>27</b>  |
| <b>3.1    Metode Pengembangan .....</b>          | <b>27</b>  |
| 3.1.1    Perancangan Sistem.....                 | 28         |
| 3.1.2    Desain Sistem .....                     | 35         |
| 3.1.3    Implementasi Sistem.....                | 47         |

|                             |                                                  |           |
|-----------------------------|--------------------------------------------------|-----------|
| 3.1.4                       | Pengujian & Evaluasi Sistem .....                | 48        |
| <b>3.2</b>                  | <b>Arsitektur AI .....</b>                       | <b>50</b> |
| 3.2.1                       | Struktur Behavior Tree .....                     | 51        |
| 3.2.2                       | Basis Pengetahuan ( <i>Knowledge Base</i> )..... | 51        |
| 3.2.3                       | Model Keputusan ( <i>Utility Score</i> ) .....   | 52        |
| <b>DAFTAR PUSTAKA .....</b> |                                                  | <b>55</b> |

## DAFTAR TABEL

|                                               |    |
|-----------------------------------------------|----|
| Tabel 2.1 Penelitian Terdahulu .....          | 24 |
| Tabel 3.1 Tabel Utility Score (Normal) .....  | 53 |
| Tabel 3.2 Tabel Utility Score (Enraged) ..... | 53 |

## DAFTAR GAMBAR

|                                                 |    |
|-------------------------------------------------|----|
| Gambar 3.1 Alur Metode Extreme Programming..... | 27 |
| Gambar 3.2 Diagram Usecase .....                | 36 |
| Gambar 3.3 Aktivitas Quest .....                | 37 |
| Gambar 3.4 Aktivitas Pertarungan.....           | 38 |
| Gambar 3.5 Sub-aktivitas Mekanisme Bos.....     | 39 |
| Gambar 3.6 Aktivitas Gacha .....                | 40 |
| Gambar 3.7 Class Diagram.....                   | 42 |
| Gambar 3.8 ERD .....                            | 43 |
| Gambar 3.9 Title Screen.....                    | 44 |
| Gambar 3.10 Home Screen .....                   | 45 |
| Gambar 3.11 Quest Selection.....                | 45 |
| Gambar 3.12 Battle Screen .....                 | 46 |
| Gambar 3.13 Gacha Screen.....                   | 46 |
| Gambar 3.14 Diagram Behavior Tree .....         | 51 |

# BAB I

## PENDAHULUAN

### 1.1 Latar Belakang

Permainan video, atau yang sering disebut game, secara fundamental adalah sebuah media interaktif yang dirancang untuk memberikan hiburan, tantangan, dan pengalaman bagi pemainnya melalui aturan-aturan tertentu dalam lingkungan virtual. Seiring perkembangan teknologi, definisi dan bentuk game terus berevolusi, mulai dari konsol klasik hingga aplikasi seluler dan platform berbasis web. Dinamika industri game yang terus berubah ini, dengan tren genre yang datang dan pergi, genre Role-Playing Game (RPG) berbasis *Turn-based* secara konsisten mampu mempertahankan basis penggemar yang loyal dan kuat. Menurut laporan Newzoo (2024), pasar game global diperkirakan akan mencapai USD 189,3 miliar pada tahun 2024, dengan RPG menjadi salah satu genre terpopuler yang mendominasi dan mendorong pertumbuhan pasar secara global, mencerminkan tingkat *engagement* yang konsisten tinggi [1]. Genre ini menawarkan kedalaman strategi, narasi yang imersif, dan mekanisme pertarungan yang membutuhkan perencanaan taktis, menjadikannya pilihan yang sesuai bagi banyak pemain.

Ketahanan genre *turn-based* ini terbukti melalui kesuksesan game-game lintas platform yang menunjukkan daya tarik universal dari genre ini. Di platform web, *Granblue Fantasy* telah menjadi rujukan sebagai RPG *turn-based* kompleks yang berhasil membangun komunitas skala global. Di platform konsol multiplatform, kesuksesan franchise *Pokemon* dengan kepopulerannya dari tahun ke tahun secara global juga menunjukkan revitalisasi genre *turn-based* dalam era lama sampai ke modern. Kesuksesan ini semakin diperkuat dengan integrasi sistem Gacha yang telah menjadi model monetisasi standar dalam

RPG modern, dimana data Sensor Tower (2024) menunjukkan bahwa mobile games bergenre RPG secara konsisten menjadi salah satu kategori teratas dalam hal pengeluaran konsumen, mencapai miliaran dolar AS dengan gacha sebagai metode penjualan mereka [2]. Pada laporan yang sama dari Newzoo (2024), retensi pemain adalah metrik kunci bagi pengembang, dan genre seperti RPG sangat unggul dalam hal ini, menunjukkan basis pemain inti untuk RPG *turn-based* memiliki tingkat retensi yang stabil [1].

Meskipun popularitas dan stabilitas genre RPG *turn-based* terbukti kuat, implementasi AI yang cerdas dan adaptif apalagi dalam platform berbasis website masih menjadi tantangan signifikan bagi industri. Menurut Chergarova, sebagian besar game RPG saat ini masih mengandalkan AI musuh yang bersifat scripted atau menggunakan pola serangan yang dapat diprediksi pemain, sehingga mengurangi tingkat tantangan dan replay value [3]. Sebuah survei pengembang game yang dilakukan oleh GDC (2024) menunjukkan bahwa banyak pengembang (84%) khawatir tentang etika penggunaan AI generatif, mengindikasikan bahwa integrasi AI canggih merupakan isu yang kompleks dan tantangan teknis serta etis bagi industri [4]. Keterbatasan ini semakin terasa dalam konteks platform web, di mana kompleksitas komputasi AI harus diseimbangkan dengan performa browser dan keterbatasan resource *client-side*.

Untuk mengatasi tantangan tersebut, penelitian ini bertujuan untuk mengembangkan *Aetheria Chronicles*, sebuah *prototype* game RPG *turn-based* berbasis web yang terinspirasi oleh mekanisme sukses dari judul-judul besar namun dengan fokus pada inovasi teknologi di balik gameplay. Penelitian ini dimaksudkan sebagai bentuk upaya dalam mengatasi keterbatasan umum game web konvensional melalui dua pembeda kunci: AI Musuh Adaptif dan arsitektur *client-server* yang kokoh. Untuk AI, penulis memilih untuk mengimplementasikan Behavior Trees (BT) yang

dikombinasikan dengan Utility Score. Behavior Trees telah terbukti sebagai pendekatan yang efektif untuk AI dalam game karena sifatnya yang modular dan mudah di-maintain [5]. Penelitian oleh Iovino dan Ögren menunjukkan bahwa Behavior Trees "were invented as a tool to enable modular AI in computer games" [6]. Dalam konteks game *turn-based* strategy, implementasi AI yang adaptif menjadi kunci untuk menciptakan pengalaman yang menantang [7] Implementasi AI adaptif dalam game modern kini menjadi standar untuk menciptakan pengalaman yang menantang, dimana AI dapat merespons strategi pemain secara dinamis daripada mengikuti pola yang dapat diprediksi. Pendekatan BT dengan Utility Score memungkinkan AI untuk mengevaluasi kondisi battlefield secara real-time seperti status debuff pada musuh, kondisi HP party pemain, atau availability skill untuk kemudian memilih strategi yang paling optimal secara dinamis, menciptakan pengalaman bertarung yang lebih variatif dan menantang. Hal ini merefleksikan praktik terbaik dalam pengembangan AI game modern dan menawarkan solusi yang lebih terukur dibandingkan sistem aturan kaku. Sementara itu, arsitektur client-server memastikan keamanan operasi Gacha yang berbasis probabilitas di backend serta menjamin persistensi data progres pemain lintas perangkat dan sesi.

Urgensi penelitian ini diperkuat oleh beberapa tantangan dan peluang di industri game web yang sejalan dengan tren genre *turn-based* yang stabil. Selain tantangan AI yang telah disebutkan, terdapat kesenjangan konten di mana game RPG berbasis web dengan sistem Senjata grid mendalam dan AI adaptif masih relatif jarang ditemukan, meskipun potensi pasarnya besar. Data dari Sensor Tower (2024) mengindikasikan bahwa genre RPG yang menggunakan sistem gacha menunjukkan pertumbuhan yang signifikan, namun dengan penawaran produk yang masih didominasi oleh segelintir judul besar [2]. Lebih lanjut, Newzoo

(2024) menunjukkan bahwa pengembang semakin berinvestasi dalam AI untuk meningkatkan personalisasi dan engagement pemain, mengindikasikan gap signifikan dalam inovasi AI game [1]. Dengan demikian, pembangunan Aetheria Chronicles tidak hanya menjadi bukti konsep teknis, tetapi juga menjadi jawaban atas kebutuhan akan game RPG yang memadukan kedalaman strategis *turn-based* dengan kecerdasan buatan yang dinamis, menawarkan pengalaman bermain yang lebih kaya dan aman dalam konteks platform web yang mudah diakses.

## 1.2 Rumusan Masalah

1. Bagaimana merancang dan membangun game RPG *Turn-based* berbasis web dengan fitur gacha dan AI adaptif?
2. Bagaimana mengembangkan sistem client-server untuk pengelolaan data pemain, gameplay, dan progres permainan yang persisten?
3. Bagaimana menerapkan sistem pertarungan *turn-based* menggunakan Behavior Tree dan perhitungan damage multi-layer dengan dukungan teknologi Phaser.js?
4. Bagaimana mengintegrasikan frontend dan backend secara fungsional agar seluruh fitur inti game dapat berjalan secara terkoordinasi dan stabil?

## 1.3 Tujuan Penelitian

1. Membangun prototipe Game RPG *turn-based* berbasis web dengan gacha dan AI adaptif.
2. Mengembangkan sisi client-server untuk mendukung pengelolaan data pemain dan gameplay yang persisten.
3. Menerapkan sistem pertarungan berbasis *turn-based* dengan pendekatan behavior trees dan perhitungan damage multi-layer menggunakan phaser.js.

4. Mewujudkan integrasi fungsional antara frontend dan backend agar seluruh fitur utama game dapat beroperasi dengan stabil.

#### **1.4 Manfaat Penelitian**

1. Menjadi acuan teknis dalam pengembangan game RPG *turn-based* berbasis web.
2. Menyediakan contoh implementasi AI adaptif menggunakan Behavior Tree dan Utility Score dalam gameplay.
3. Menunjukkan penerapan sistem gacha dan mekanisme damage kompleks di platform web.
4. Memberikan gambaran penerapan arsitektur client-server dalam pengelolaan data game secara persisten.

#### **1.5 Batasan Penelitian**

1. Platform Pengembangan  
Game akan dikembangkan sebagai Game berbasis web yang berjalan di sisi klien maupun server. Pada bagian frontend/sisi klien akan menggunakan Phaser, sedangkan lingkungan backend/sisi server akan dibangun dengan Node.js dan Express.js, didukung oleh MySQL sebagai *database*.
2. Skala Konten
  - a. Koleksi Karakter  
6 Karakter SSR, 8 Karakter SR (yang didapat otomatis bersama senjata).
  - b. Koleksi Senjata  
6 Senjata SSR, 12 Senjata SR, dan 20 Senjata R.
  - c. Quest

8 quest utama (untuk progres cerita) dan 3 impossible quest (untuk tantangan akhir atau *endgame content*).

3. Implementasi AI

Penerapan Behavior Tree (BT) hanya akan difokuskan pada pengambilan keputusan skill oleh musuh (NPC) dalam sesi pertarungan. Target waktu pemrosesan keputusan AI adalah maksimal 500ms per giliran musuh.

4. Target Performansi

Game ditargetkan untuk mencapai minimal 30 FPS (*Frames Per Second*) pada perangkat low-end (spesifikasi contoh: Quad-core 1.5GHz, RAM 2GB, peramban Google Chrome).

5. Fitur Multiplayer

Proyek ini tidak akan mencakup implementasi fitur real-time PvP (*Player versus Player*). Interaksi pemain hanya terbatas pada sesi *single-player* atau PvE (*Player versus Environment*) dengan data persisten.

6. Fokus Balancing Game

Proses balancing nilai damage, *drop rate*, gacha, dan tingkat kesulitan AI akan dilakukan melalui pengujian iteratif, namun tidak menjadi fokus penelitian utama dalam konteks pengembangan algoritma Machine Learning untuk balancing otomatis.

## **BAB II**

### **KAJIAN PUSTAKA**

#### **2.1 Kajian Teori**

##### **2.1.1 Konsep Dasar Pengembangan**

###### **a. Game RPG Turn Based**

Permainan video, atau yang umum disebut game, secara fundamental adalah sebuah bentuk media interaktif yang dirancang untuk memberikan pengalaman bermain bagi partisipannya melalui seperangkat aturan. Menurut Schell pada bukunya yang berjudul "*The Art of Game Design*" menegaskan, inti dari sebuah game melibatkan adanya tujuan, aturan, tantangan, dan interaksi yang bermakna antara pemain dengan sistem atau pemain lain. Schell juga menegaskan bahwa "mainan adalah objek yang kamu mainkan", yang membedakan game dari mainan karena game memiliki kompleksitas dan jenis permainan yang lebih terstruktur [8]. Lebih dari sekadar hiburan pasif, game menawarkan sebuah sistem di mana pemain mengambil keputusan, menghadapi konsekuensi, dan beradaptasi dalam lingkungan virtual yang terstruktur. Ini membedakan game dari bentuk media lain seperti film atau buku, di mana interaksi pemain adalah elemen sentral dalam membentuk pengalaman yang disajikan. Penting untuk dipahami bahwa "game bukanlah pengalaman itu sendiri. Game memungkinkan pengalaman itu terjadi, tetapi game bukan pengalaman itu" [8]. Oleh karena itu, fokus utama desainer game adalah pada pengalaman yang dirasakan oleh pemain.

Seiring berjalannya waktu, definisi game telah berevolusi dengan kemajuan teknologi dan munculnya genre-genre baru. Dalam spektrum luas genre game, RPG

*Turn-based* menonjol sebagai perpaduan antara elemen naratif yang kaya dari *Role Playing Game* (RPG) dengan mekanisme strategi berbasis giliran (*turn-based*). Menurut David Vincent Turangan, RPG adalah subgenre dari *action* dimana pemain memainkan game dari sudut pandang salah satu karakter utama dalam petualangannya, sementara *turn-based* adalah permainan dengan pertarungan yang bergantian antara beberapa karakter atau AI [9]. Hal ini memberikan pemain waktu untuk mempertimbangkan dengan teliti setiap langkah dan strategi terbaik untuk mengalahkan lawan.

Penggabungan kedua genre ini menghasilkan game dengan *gameplay* di mana pemain akan berperan sebagai seorang karakter dalam petualangannya, dan saat pertarungan, pemain akan melawan musuh secara bergantian [9]. Game RPG *Turn-based* seperti *Pokémon Sword and Shield* atau *Octopath Traveller* menuntut kemampuan berpikir logis dan strategis dari pemain, menjadikan setiap pertarungan sebagai tantangan yang mendalam dan membutuhkan perhitungan matang. Interaksi inilah yang membentuk pengalaman bermain, mendorong pemain untuk menguasai sistem, mengatasi rintangan, dan mencapai tujuan yang ditetapkan dalam dunia game.

b. AI dalam Game

Kecerdasan buatan (*Artificial Intelligence/AI*) dalam game adalah seperangkat algoritma untuk mengendalikan perilaku *Non-Playable Characters* (NPC) agar dapat merespons kondisi permainan secara dinamis. Modern game AI tidak lagi sekadar *reactive systems*, melainkan *proactive agents* yang dapat menganalisis game state, memprediksi perilaku pemain, dan menyesuaikan strategi

secara *real-time*. Marcotte R. (2017) menyebut bahwa tantangan utama bagi pengembang adalah menciptakan perilaku karakter dan objek yang menarik, dengan keseimbangan antara efisiensi komputasi dan realisme perilaku [10].

Dalam game *turn-based*, AI berperan penting dalam mengatur strategi serangan, pertahanan, dan penggunaan kemampuan khusus. Sekhavat (2017) menegaskan bahwa AI efektif harus mempertimbangkan psikologi pemain dan tingkat keterlibatan, karena AI yang terlalu dominan menimbulkan frustrasi, sedangkan yang terlalu mudah diprediksi mengurangi replay value [5]. Desain AI yang efektif tidak hanya mempertimbangkan aspek teknis seperti *pathfinding* atau *rule-based decision making*, tetapi juga memastikan tingkat kesulitan yang seimbang sehingga pemain terdorong untuk mengembangkan strategi yang adaptif.

Pengembangan AI modern menghadapi tantangan menciptakan lawan yang menantang namun adil. Apperson (2024) menemukan bahwa kepuasan pemain terhadap AI sangat dipengaruhi oleh kecerdasan yang terkesan konsisten dan tidak berlebihan, serta kemampuan AI untuk menunjukkan strategi yang tidak mudah ditebak.

c. Mekanik Gacha

Sistem monetisasi gacha dalam game online multiplayer telah mengalami evolusi signifikan, mengubah desain game dan pengalaman pemain [11]. Mekanik gacha, yang berasal dari mesin penjual mainan "gachapon" Jepang, memotivasi pemain untuk menggunakan mata uang dalam game atau uang

sungguhan demi kesempatan mendapatkan hadiah virtual secara acak, seperti karakter atau kosmetik [11].

Andriyanto menegaskan pada penelitiannya tentang bagaimana mekanik gacha bekerja. Penyebab kegembiraan pemain dalam game gacha ini berakar pada beberapa faktor. Salah satunya adalah adanya "harapan untuk pengumuman" atau Expectation for Announcement [12]. Pemain merasa lebih baik dan lebih termotivasi untuk melakukan gacha jika mereka mengetahui persentase kemungkinan mendapatkan karakter atau item. Hal ini menunjukkan bahwa transparansi peluang, meskipun rendah, dapat meningkatkan inisiatif pemain untuk "menarik/pull/roll" atau melakukan gacha demi mendapatkan karakter atau item tertentu. Perasaan ini juga didukung oleh keyakinan bahwa suatu saat mereka pasti akan mendapatkan karakter atau item yang diinginkan dari gacha, meskipun peluangnya kecil [12]

Selain harapan, "kepuasan pengumuman" atau Satisfaction of Announcement juga berperan. Meskipun dampaknya lebih kecil dibandingkan harapan, kepuasan terhadap informasi persentase kemungkinan yang diberikan dapat memengaruhi pemain. Jika pemain merasa persentase yang diberitahukan cukup akurat dan mereka puas dengan hal tersebut, ini dapat berkontribusi pada pengalaman bermain mereka [12]. Kombinasi harapan dan kepuasan ini mendorong pemain untuk terus mencoba gacha, dan ketika mereka berhasil mendapatkan item yang diinginkan, mereka merasakan kesenangan yang lebih besar dalam gameplay. Sensasi kegembiraan ini bahkan dapat meningkatkan kemungkinan pemain menjadi kecanduan sementara atau "berjudi" dengan

gacha, meskipun persepsi dan cara menikmati konten game dapat bervariasi antar pemain. Oleh karena itu, implementasi sistem gacha yang efektif sangat penting untuk memastikan kompatibilitas dengan jenis game dan tujuan perusahaan [12].

d. Studi Kasus: *Granblue Fantasy*

*Granblue Fantasy* adalah sebuah *Role-Playing Game* (RPG) *turn-based* berbasis web yang dikembangkan oleh Cygames dan dirilis pada tahun 2014. Game ini merupakan game *online free-to-play* untuk browser dan perangkat mobile yang telah menjadi salah satu *franchise* multimedia terbesar Jepang dengan ekspansi ke berbagai media termasuk anime dan game spin-off. Menurut penelitian Deris Stiawan pada 2020, *Granblue Fantasy* adalah salah satu game dengan genre RPG *turn-based* yang berhasil membuktikan viabilitas platform web untuk menghadirkan pengalaman RPG yang kompleks [13].

Keunikan utama *Granblue Fantasy* terletak pada sistem weapon grid yang rumit, dimana pemain dapat mengkustomisasi kekuatan karakter melalui kombinasi senjata dan summon monster yang kompleks. Game ini menggunakan mekanisme *turn-based* klasik dengan elemen strategi melalui sistem elemen, buff/debuff, dan skill yang membutuhkan perencanaan taktis. Sistem gacha dalam *Granblue Fantasy* juga menjadi model yang berhasil mengintegrasikan monetisasi dengan *progression system* yang memberikan value jangka panjang bagi pemain. Official website *Granblue Fantasy* memberikan informasi mendalam mengenai game ini, seperti informasi seputar update game, latar dunia, karakter dan mekanisme permainan, serta orang-orang dibalik pembuatan game ini

sendiri [14]. Hal ini menunjukkan bahwa developer fokus pada world-building yang imersif.

Kesuksesan *Granblue Fantasy* terbukti melalui ekspansi franchise yang luas, salah satunya yaitu *Granblue Fantasy: Relink* yang dirilis untuk platform PC dengan genre Action RPG Hack and Slash dan sudah dirilis pada Februari 2024 kemarin. Informasi ini mendemonstrasikan kekuatan IP yang dibangun dari game RPG turn-based orisinal dari platform browser mampu menunjukkan bahwa fondasi konsep dan gameplay yang solid dapat menjadi basis untuk pengembangan franchise multimedia yang sukses.

### **2.1.2 Mekanisme Pertarungan**

#### **a. Sistem Turn Order/Queue**

Sistem turn order pada *Aetheria Chronicles* menerapkan *fixed turn order*, di mana giliran pemain selalu dimulai terlebih dahulu setiap ronde. Pendekatan ini selaras dengan temuan Aranda et al. (2020) yang menunjukkan bahwa pemain cenderung lebih menyukai *turn-based systems* karena memberikan kontrol yang lebih baik dalam pengambilan keputusan taktis dibanding *real-time systems* [15]. Karakteristik ini juga identik dengan JRPG klasik seperti *Final Fantasy* dan *Dragon Quest*, yang memungkinkan perencanaan strategi sebelum eksekusi [16].

Selama giliran pemain, seluruh karakter dalam *party* dapat menggunakan keterampilan (*Skill*) yang tidak berada dalam status *cooldown*. Indikator giliran ditampilkan melalui *highlight* pada karakter aktif, teks "*Player Turn*", dan tombol *Attack* untuk mengakhiri giliran. Setelah itu, giliran musuh berlangsung dengan pola serangan bergantian untuk setiap unit. Baik pemain

maupun musuh memiliki *Special Attack (SA) Bar* yang terisi melalui *basic attack* atau efek *skill* tertentu. Saat terisi penuh, *Special Attack* dapat dieksekusi manual atau otomatis. Penggunaan *Special Attack* secara bersamaan oleh beberapa karakter memicu *SA Burst* dengan bonus kerusakan hingga 400%, mengadopsi konsep *combo system* yang terbukti efektif meningkatkan kedalaman strategi dan memberi insentif koordinasi antar karakter.

b. Sistem *Skill* dan *Cooldown*

Setiap karakter memiliki empat *skill* aktif yang terbagi dalam tiga kategori utama: keterampilan serangan (*damage skill*), peningkatan kemampuan (*buff/heal skill*), dan penurunan kemampuan musuh (*debuff skill*). Klasifikasi ini mengikuti konsep *trinity system* yang umum digunakan dalam desain RPG modern, di mana keseimbangan antara *damage*, *support*, dan *control* menjadi faktor utama keberhasilan strategi tim [17]. Penelitian Amereh (2024) ini menegaskan bahwa RPG umumnya menawarkan “beragam pilihan kelas karakter dan kemampuan yang menciptakan pengalaman tempur yang kaya dan dinamis,” terutama pada sistem berbasis giliran (*turn-based*) yang memiliki “set dan efek kemampuan yang lebih kompleks” [17].

Setiap keterampilan memiliki durasi *cooldown* yang bervariasi tergantung kekuatan dan efeknya. Misalnya, keterampilan dengan *damage output* tinggi akan memiliki *cooldown* lebih panjang untuk mencegah penggunaan berulang. Aspek *cooldown* ini sejalan dengan *Gameplay Ability System (GAS)* yang mengelola “pengaktifan, eksekusi, dan *cooldown* kemampuan” [17], sehingga menjadi elemen krusial dalam permainan strategis karena menuntut pemain menimbang efektivitas langsung dan

ketersediaan kemampuan di masa depan. Beberapa efek *Special Attack* atau *skill* yang dimiliki karakter dapat memperpendek *cooldown*, menciptakan variasi strategi sebagaimana dijelaskan dalam *TTRPG Combat Balancing Guide* (2025), di mana manipulasi *cooldown* menjadi inti dari *high-level play* [18].

c. Sistem Perhitungan Damage Multilayer

Sistem *multilayer damage calculation* di *Aetheria Chronicles* membagi proses perhitungan kerusakan menjadi beberapa tahap yang saling memengaruhi. Pendekatan ini mengikuti *best practices* pengembangan RPG, yang memungkinkan penyesuaian keseimbangan permainan (*fine-tuning*) serta memberikan transparansi kepada pemain terkait sumber kerusakan. Tahap pertama adalah menghitung *Raw Damage*, yaitu gabungan atribut serangan karakter dan akumulasi atribut dari *weapon grid*, kemudian dimodifikasi oleh persentase peningkatan atribut berdasarkan level karakter. Sebagai perbandingan, sistem *Dungeons & Dragons* (D&D) menerapkan *attack roll* dengan penambahan *modifier* dari atribut, kemampuan, peralatan, dan efek status, yang mencerminkan adanya perhitungan berlapis [17].

Tahap berikutnya menghasilkan *Final Damage* dengan mempertimbangkan *weapon skill*, kecocokan elemen (*element matching*), serta pengganda seperti *critical hit* atau *multi-attack*. Nilai akhir atau *Output Damage* diperoleh setelah menerapkan efek *buff* dan *debuff*, termasuk keuntungan *element counter*. Konsep ini selaras dengan *Gameplay Effects System* (GES), yang mampu “mengubah atribut dan *gameplay tags* pada diri sendiri maupun pihak lain,” serta “mendefinisikan modifikasi atribut seperti penerapan *damage*, *buffs*, atau *debuffs*” [17]. Pendekatan ini

menekankan struktur perhitungan berlapis untuk menghasilkan nilai kerusakan yang adil, seimbang, namun tetap memuaskan secara gameplay.

d. Sistem Elemen dan *Buff/Debuff*

Sistem elemen mencakup tiga tipe utama Api, Angin, dan Tanah yang saling berinteraksi dalam pola counter melingkar (Api > Angin > Tanah > Api). Mekanisme *rock-paper-scissors* ini mengikuti prinsip desain permainan yang mudah dipahami namun memiliki kedalaman strategi yang cukup. Keunggulan *element counter* memberi bonus kerusakan serta pengurangan kerusakan diterima, menciptakan pilihan yang bermakna dalam penyusunan tim dan strategi pertempuran. Konsep ini sejalan dengan sistem *advantage/disadvantage* elemen dalam *Tactical RPG*, yang memengaruhi interaksi karakter dalam narasi [16].

Selain elemen, pertarungan juga dipengaruhi oleh efek *buff* dan *debuff*. *Buff* dapat meningkatkan atribut seperti serangan, pertahanan, atau laju pengisian *Special Attack* dan *Cooldown skill*, sedangkan *debuff* menurunkan kemampuan lawan atau memberikan efek negatif seperti pengurangan status, pengisian *Special Attack*, dan efek *Crowd Control* seperti *stun*, dan *dispel*. Sinergi antara mekanisme elemen dan efek status ini menciptakan *decision space* yang luas dan memberikan penghargaan strategis bagi pemain yang mampu memanfaatkannya secara optimal.

e. Manajemen Data Game

Manajemen data adalah aspek krusial dalam pengembangan game, terutama untuk memastikan keberlanjutan progres pemain dan mendukung game analytics. Data ini mencakup informasi penting seperti

progres quest, inventaris, level karakter, dan atribut, yang merupakan bagian dari player metrics dan player behaviors yang dikumpulkan untuk analisis [19]. Sistem manajemen data dalam game umumnya mengadopsi arsitektur client-server untuk menjamin konsistensi data lintas perangkat dan sesi bermain, di mana data pemain yang sensitif dan inti disimpan secara terpusat di server. Pendekatan ini esensial untuk game modern yang membutuhkan persistensi data dan interaksi multi-device, karena menyediakan sumber data tunggal yang terpercaya [19].

Menurut Su et al. (2021), data perilaku pemain seperti posisi karakter, health, mana, stamina, nama karakter, level, perlengkapan, dan mata uang dapat dikumpulkan dari klien game dan server game. Data ini menjadi dasar untuk player analytics yang bertujuan mengidentifikasi pola perilaku pemain dan mendukung pengambilan keputusan dalam desain game [19]. Sementara mekanisme penyimpanan sisi klien seperti `localStorage` atau `IndexedDB` dapat dimanfaatkan untuk data bersifat sementara atau preferensi pengguna lokal, data inti seperti progres karakter dan aset yang diperoleh melalui gacha harus disimpan di sisi server untuk mencegah manipulasi data dan menjaga integritas permainan. Oleh karena itu, penerapan arsitektur client-server memungkinkan implementasi fitur penting seperti auto-save, permainan lintas perangkat, tracking analytics untuk memahami player churn dan pendapatan, serta mekanisme anti-cheat yang krusial untuk pemeliharaan dan optimalisasi game jangka panjang [19].

### 2.1.3 Kecerdasan Buatan (AI) dalam Game

#### a. Behavior Trees dan Utility Score

Behavior Tree (BT) adalah kerangka kerja untuk memodelkan logika pengambilan keputusan AI secara hierarkis. BT memecah perilaku kompleks menjadi node-node yang saling terhubung, dengan *control flow* yang mengatur urutan evaluasi kondisi dan eksekusi aksi. Menurut Iovino & Ögren (2022), "*Behavior Trees (BT) were invented as a tool to enable modular AI in computer games, but have received an increasing amount of attention in the robotics community*" karena kemampuannya membangun sistem perilaku yang kompleks [6]. Ögren (2022) menegaskan bahwa BT adalah cara efisien untuk mengatur pergantian tugas pada agen otonom, baik robot maupun entitas virtual, dengan sifat modular dan reaktif yang memungkinkannya merespons perubahan game *state* secara dinamis [20].

Dalam pengembangan game modern, BT populer karena modular, skalabel, mudah diuji, dan dapat diatur prioritasnya [20]. Namun, BT murni cenderung deterministik, AI akan memilih aksi pertama yang memenuhi kondisi sehingga perilaku bisa menjadi mudah diprediksi pemain. BT yang diimplementasikan secara murni, akan memunculkan sifat deterministik karena *selector node* dirancang untuk memproses komponen anak dari kiri ke kanan dan segera berhasil setelah menemukan opsi yang valid pertama. Pendekatan "opsi valid pertama" ini berarti AI akan selalu mengeksekusi aksi yang sama ketika kondisi yang sama terpenuhi [5]. Selain itu, sintaks standar BT tidak memiliki fitur bawaan untuk perilaku probabilistik, yang semakin

memperkuat sifat deterministik dan berpotensi membuat pola perilaku AI mudah diantisipasi oleh pemain.

Untuk mengatasi keterbatasan ini, BT akan dikombinasikan dengan *Utility Score*, yaitu metode penilaian berbasis skor yang memilih aksi dengan nilai tertinggi pada situasi tertentu. Konsep ini berakar pada *utility theory* dalam ekonomi dan *decision science*, di mana Rudramala (2024) menekankan bahwa "*Decision making forms the core of any AI system*" [21]. McGuire (2019) menjelaskan bahwa dalam implementasi *utility AI*, sistem memberi skor pada setiap aksi potensial berdasarkan *current world state*, lalu mengeksekusi yang bernilai paling tinggi, sehingga perilaku AI menjadi lebih dinamis dan kontekstual [22].

Dalam konteks *Aetheria Chronicles*, skor *utility* dihitung dari variabel seperti persentase *HP(Health Points)*, jumlah *buff/debuff*, *threat level*, hingga status *Special Attack Bar*. Kombinasi BT dan *Utility Score* ini mempertahankan struktur logika yang rapi sekaligus fleksibilitas memilih aksi optimal sesuai situasi pertarungan, menghasilkan AI yang terstruktur namun adaptif – menggabungkan keandalan arsitektur BT dengan fleksibilitas pengambilan keputusan berbasis *utility*.

### **2.1.4 Arsitektur Sistem Berbasis Web**

#### **a. Arsitektur Client-Server**

Arsitektur *client-server* adalah paradigma fundamental dalam pengembangan aplikasi web yang memisahkan tanggung jawab antara *client (frontend)* dan *server (backend)*. Arsitektur ini memungkinkan banyak pengguna untuk secara bersamaan mengakses dan menggunakan sumber daya yang sama [23]. *Client* menyediakan antarmuka yang ramah pengguna untuk

meminta sumber daya dari server. Server kemudian menerima permintaan, memprosesnya, dan mengembalikan respons ke client yang meminta [23].

Dalam konteks game web, pemisahan ini memungkinkan client untuk menangani rendering grafis dan interaksi pengguna, sementara server bertanggung jawab atas logika bisnis, validasi data, dan persistensi informasi game. Keunggulan arsitektur client-server meliputi penyimpanan data terpusat dengan akses jarak jauh dari komputer client yang tersebar secara geografis, skalabilitas yang lebih baik karena lebih banyak sumber daya dapat dengan mudah ditambahkan, dan pemeliharaan sumber daya yang lebih mudah dan murah karena dapat dikelola secara terpusat [23]. Selain itu, arsitektur ini memungkinkan penyeimbangan beban kerja (load balancing) dengan menambahkan server redundan dan replika untuk menyebarkan beban kerja dan menawarkan opsi pemulihan yang lebih mudah jika terjadi kegagalan server [23]. Implementasi arsitektur client-server dalam game web, terutama dalam penanganan operasi yang membutuhkan konsistensi data seperti sistem gacha, inventory management, dan progress permainan dapat meningkatkan performa karena data dan logika bisnis dipisahkan, dan setiap server atau tier fokus pada pemrosesan permintaan intinya [23].

b. Phaser (*Frontend/Client Side*)

Phaser.js merupakan framework JavaScript open-source untuk pengembangan game 2D berbasis web dengan dukungan HTML5 Canvas dan WebGL. Framework ini menyediakan fitur terintegrasi seperti sistem animasi, physics engine, manajemen audio, dan

pengelolaan input, sehingga menjadi solusi komprehensif untuk game development [24]. Keunggulan utamanya terletak pada fleksibilitas platform dan kemudahan implementasi, memungkinkan game dibangun untuk berbagai platform seperti web, YouTube Playables, Discord Activities, Twitch Overlays, hingga aplikasi native di iOS, Android, dan Steam menggunakan tools pihak ketiga. Phaser mendukung JavaScript maupun TypeScript, memiliki kompatibilitas lintas platform dan mengotomatiskan banyak tugas grafis serta input handling yang jika dilakukan manual akan memakan waktu dan tenaga [25]. Hal ini memungkinkan pengembang lebih fokus pada gameplay dan logika inti game dibanding detail implementasi grafis tingkat rendah.

Dalam konteks pengembangan game RPG turn-based seperti Aetheria Chronicles, Phaser.js menjadi fondasi teknologi yang solid untuk mengimplementasikan gameplay kompleks. Framework ini mendukung integrasi dengan lebih dari 40 framework frontend seperti React dan Vue, secara otomatis memilih metode rendering optimal antara WebGL dan Canvas untuk menjaga performa pada berbagai konfigurasi perangkat keras dan peramban, serta menyediakan dukungan fitur seperti sprite animation, tilemaps, audio, dan beragam physics engine [25]. Stabilitasnya, dengan pengembangan aktif selama lebih dari 11 tahun, didukung komunitas besar di GitHub yang responsif terhadap perbaikan bug dan diskusi praktik terbaik [24]. Hal menjadikan Phaser pilihan tepat untuk pengembangan game 2D yang kompleks dan berjangkauan luas berbasis web.

- c. Node.js dan Express.js (*Backend/Server Side*)

Node.js, sebagai JavaScript runtime environment menawarkan keunggulan dalam menjaga konsistensi bahasa pemrograman antara frontend dan backend, yang secara signifikan mengurangi kompleksitas dalam pengembangan dan pemeliharaan aplikasi [26]. Keunggulan ini sangat relevan dalam pengembangan game RPG Turn-based yang seringkali melibatkan logika kompleks di kedua sisi. Node.js juga mengadopsi arsitektur event-driven dan single-threaded, yang meskipun tampak seperti langkah mundur, justru merupakan terobosan cerdas karena sangat menyederhanakan penulisan aplikasi web dan sangat sesuai untuk aplikasi game yang membutuhkan penanganan concurrent requests dan pemrosesan data real-time [26]. Dalam skenario high-concurrency seperti game online, di mana banyak pemain dapat berinteraksi secara simultan, kemampuan Node.js untuk menangani banyak koneksi tanpa blocking sangat krusial untuk menjaga kelancaran pengalaman bermain. Selain itu, sifat platform-independent dari Node.js memungkinkan kolaborasi yang mudah di antara tim pengembang yang mungkin menggunakan sistem operasi yang berbeda [26].

Melengkapi Node.js, Express.js berfungsi sebagai web application framework minimalis dan fleksibel yang menyediakan fitur lengkap untuk aplikasi web, termasuk dukungan middleware, mekanisme routing yang kuat, dan pengembangan RESTful API [26]. Kombinasi Node.js dan Express.js terbukti efektif dalam menangani skenario high-concurrency dengan kemampuan menangani banyak koneksi simultan secara efisien [26]. Dalam konteks game RPG Turn-based dengan fitur Gacha dan AI Adaptif, arsitektur ini memungkinkan implementasi

pembuatan angka acak yang aman (secure random number generation) untuk mekanisme Gacha dan pembaruan inventaris real-time dengan latensi minimal. Fitur routing Express.js yang canggih juga memungkinkan penanganan kueri kompleks untuk analisis statistik game dan sistem papan peringkat, serta mendukung berbagai metode HTTP yang diperlukan untuk interaksi API yang dinamis. Fleksibilitas Express.js memungkinkan pengembang untuk menambahkan fungsionalitas yang diperlukan saja, menghindari bloatware yang tidak perlu, sehingga menghasilkan proyek yang lebih ringan dan efisien [26].

d. MySQL (Database)

MySQL merupakan relational database management system (RDBMS) yang menyediakan kepatuhan ACID (Atomicity, Consistency, Isolation, Durability) dan dukungan transaksi yang kuat [27]. Hal ini sangat penting untuk menjaga integritas data dalam sistem game RPG Turn Based, khususnya dalam penyimpanan data pemain seperti progres quest, informasi karakter, dan item. Konsistensi data menjadi krusial dalam game, terutama untuk progres karakter, manajemen inventaris, dan transaksi gacha, di mana setiap operasi harus dipastikan berhasil sepenuhnya atau gagal seluruhnya untuk menghindari anomali data [27].

Selain itu, MySQL mendukung kueri kompleks dan joins yang esensial untuk analisis statistik game dan sistem papan peringkat. Jurnal ini menekankan bahwa RDBMS seperti MySQL, melalui pengoptimalan kueri dan pemanfaatan indeks yang tepat, mampu menangani transaksi dengan efisien. Hal ini sangat relevan untuk skenario high-concurrency dalam game, di mana banyak

pemain mungkin mengakses atau memodifikasi data secara bersamaan. Pengoptimalan eksekusi kueri melalui teknik seperti query rewriting, pengindeksan, dan caching dapat secara signifikan mengurangi waktu respons kueri dan meningkatkan throughput sistem secara keseluruhan [28]. Dengan memanfaatkan teknik ini, administrator basis data dan pengembang dapat menyempurnakan rencana eksekusi kueri, meminimalkan kebutuhan akan operasi yang intensif sumber daya, dan meningkatkan efisiensi sistem secara keseluruhan [28].

Keamanan data juga merupakan pertimbangan utama. MySQL menyediakan fitur enkripsi dan manajemen hak istimewa pengguna yang penting untuk melindungi data pemain yang sensitif dan mencegah upaya kecurangan, terutama dalam sistem berbasis probabilitas seperti mekanisme gacha [28]. Untuk memastikan performa optimal, pemantauan dan penyetelan kinerja secara berkelanjutan juga diperlukan. Hal ini melibatkan pelacakan metrik penting seperti waktu eksekusi kueri, penggunaan CPU, dan tingkat I/O disk untuk mengidentifikasi bottleneck dan melakukan penyesuaian konfigurasi basis data yang tepat [28].

## 2.2 Penelitian Terdahulu

Kajian ini mencakup berbagai aspek yang menjadi fondasi pengembangan *Aetheria Chronicles*, mulai dari desain sistem pertarungan *turn-based* dan mekanisme *gacha*, implementasi kecerdasan buatan (AI) seperti *Monte Carlo Tree Search* (MCTS), hingga arsitektur teknis pendukung seperti pengembangan berbasis web dan model *client-server*. Tabel berikut akan merangkum fokus utama, hasil, dan celah penelitian (*research gap*)

dari setiap studi untuk memberikan konteks terhadap kebaruan dan kontribusi dari penelitian ini.

Detail ringkasan relevansi dan gap dari masing-masing penelitian dapat dilihat pada Tabel 2.1 berikut.

Tabel 2.1 Penelitian Terdahulu

| Peneliti dan Judul                                                                 | Hasil Penelitian                                                                                                                                 | Gap Penelitian                                                                                                                  |
|------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Adrianus (2021) - Implementasi BT pada NPC di game Sidescroller [29]               | BT efektif membentuk perilaku NPC adaptif                                                                                                        | Genre berbeda (sidescroller); belum mencakup <i>turn-based</i> , RPG mekanik, dan penerapan BT + utility score                  |
| Fatemeh (2024) - Implementation of <i>Turn-based</i> Combat Systems in RPGs [17]   | Desain modular logika <i>turn-based</i>                                                                                                          | Tidak spesifik ke web stack/Phaser, tidak membahas gacha & penerapan AI adaptif                                                 |
| Bunga P. I. (2024) - Factors That Influence Gacha Game Users' Decision-Making [30] | Menemukan bahwa niat pemain menggunakan game <i>gacha</i> secara signifikan dipengaruhi oleh ekspektasi performa, motivasi hedonis (kesenangan), | Fokus pada aspek psikologis pengguna. Tidak membahas perancangan atau implementasi teknis dari sistem <i>gacha</i> itu sendiri. |

| Peneliti dan Judul                                                                                   | Hasil Penelitian                                                                                | Gap Penelitian                                                                                                                                 |
|------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                      | dan kebiasaan ( <i>habit</i> ).                                                                 |                                                                                                                                                |
| Marcotte & Hamilton (2017) - Behavior Trees for Modelling Artificial Intelligence [31]               | AI meningkatkan efisiensi dev & <i>engagement</i> pemain                                        | Cakupan luas; tidak spesifik ke BT/utility dan <i>turn-based</i>                                                                               |
| Solihin (2023) - Game Ular Tangga Berbasis Web [32]                                                  | Game web fungsional via browser                                                                 | Game Web sederhana; tanpa AI adaptif, gacha, kompleksitas RPG tidak dibahas                                                                    |
| Yuka (2018) - Adaptive AI pada Game Turn Based RPG Dengan Metode <i>Monte Carlo Tree Search</i> [33] | MCTS terbukti efektif dengan rata-rata kemenangan 83% melalui simulasi prediktif.               | Menggunakan pendekatan AI yang berbeda ( <i>simulation-heavy</i> MCTS).                                                                        |
| Karsten (2024) - Perancangan Game Turn Based 'Elemental Fight' pada HTML5 [34]                       | Berhasil membangun game <i>turn-based</i> untuk platform HTML5 menggunakan Unity, yang mencakup | Menggunakan <i>engine</i> berbeda (Unity) dan tidak secara spesifik membahas implementasi AI yang kompleks seperti BT + <i>Utility Score</i> . |

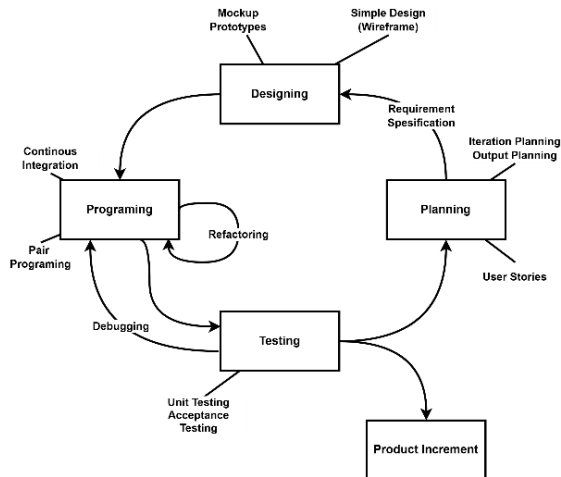
| Peneliti dan Judul | Hasil Penelitian                      | Gap Penelitian |
|--------------------|---------------------------------------|----------------|
|                    | fitur <i>stage</i> dan <i>gacha</i> . |                |

Berdasarkan tinjauan pada Tabel 2.1, dapat disimpulkan bahwa penelitian-penelitian sebelumnya telah memberikan landasan yang kuat pada setiap komponen individu dari proyek ini. Studi tentang AI adaptif, mekanisme game, serta arsitektur teknis berbasis web telah banyak dilakukan secara terpisah. Namun, celah penelitian (*research gap*) yang signifikan muncul dari kurangnya studi yang mengintegrasikan semua elemen ini – AI adaptif berbasis BT dan *Utility Score*, mekanisme *gacha*, dan sistem pertarungan *turn-based* yang kompleks ke dalam satu prototipe game berbasis web yang kohesif. Oleh karena itu, penelitian ini bertujuan untuk mengisi celah tersebut dengan merancang dan membangun *Aetheria Chronicles* sebagai sebuah studi kasus implementasi yang komprehensif.

### BAB III

### 3.1 Metode Pengembangan

Metode *Extreme Programming* (XP) dipilih untuk pengembangan Aetheria Chronicles karena kebutuhannya akan siklus pengembangan yang cepat dan adaptif, yang sangat cocok untuk proyek game modern dengan kebutuhan yang dapat berubah secara dinamis. Proyek ini melibatkan komponen kompleks seperti mekanisme pertarungan dan AI, di mana XP memungkinkan pembuatan prototipe cepat dan pengujian fitur sejak awal untuk meminimalkan risiko. Hal ini sejalan dengan pernyataan Dovetail (2024) bahwa XP sangat efektif untuk proyek dengan kebutuhan yang berevolusi cepat dan menuntut kolaborasi erat antara tim dan stakeholder [36]. Pendekatan iteratif ini juga krusial untuk pengembangan game yang memerlukan penyeimbangan dan optimalisasi berkelanjutan, sejalan dengan penekanan XP pada umpan balik berkelanjutan dan adaptasi untuk mengoptimalkan pengalaman pengguna.



Gambar 3.1 Alur Metode *Extreme Programming*

Dalam penelitian ini penerapan metode XP akan dimulai dari tahapan pertama Perancangan Sistem (*planning*) lalu dilanjutkan dengan Desain Sistem (*Designing*) lalu diimplementasikan dalam bentuk program (*Programming*) dan disambung dengan tahapan terakhir yakni pengujian sistem (*testing*) yang telah dikembangkan dari tahapan sebelumnya. Setiap tahap diterapkan secara iteratif dan berfokus pada kolaborasi, umpan balik cepat, serta fleksibilitas terhadap perubahan kebutuhan selama proses pengembangan game *Aetheria Chronicles*. Penerapan XP menghasilkan peningkatan signifikan dalam hal pemeliharaan dan skalabilitas sistem dibandingkan dengan metode tradisional [37]. Dengan demikian, penerapan metode XP dapat memberikan kerangka kerja yang solid untuk pengembangan yang fleksibel dan responsif terhadap perubahan, tanpa mengorbankan kualitas akhir produk, sebagaimana yang ditekankan oleh A.Sasongko [38].

### **3.1.1 Perancangan Sistem**

Tahap perancangan sistem ini dimulai dengan Analisis Kebutuhan Sistem. Dalam konteks *Extreme Programming* (XP), tahap ini merupakan tahapan fundamental dari fase *planning* untuk memastikan seluruh fitur selaras dengan visi proyek *Aetheria Chronicles*.

Dalam konteks XP, fase Planning juga mencakup penyusunan user stories yang menggambarkan perilaku sistem dari sudut pandang pemain, seperti fitur pertarungan turn-based, sistem gacha, manajemen party, dan inventory. Hasil dari tahap ini menjadi dasar bagi tahapan desain berikutnya, dengan tujuan menciptakan sistem yang mudah dikembangkan secara iteratif dan mampu beradaptasi terhadap perubahan kebutuhan selama proses pengembangan.

- a. Analisis Kebutuhan Sistem
  1. Kebutuhan Fungsional
    - a) *User Account & Login*

- i. Sistem harus menyediakan fitur pendaftaran akun baru.
  - ii. Sistem harus menyimpan dan memvalidasi data login pemain.
  - iii. Sistem harus mengelola sesi pengguna yang aktif setelah login berhasil untuk menjaga status login di seluruh sistem.
- b) Mekanisme Pertarungan
  - i. Pemain dapat memilih dan menggunakan skill saat gilirannya.
  - ii. Sistem harus menghitung damage dengan perhitungan multilayer.
  - iii. Sistem harus mendukung keutungan elemen counter dan buff/debuff.
  - iv. Sistem harus mengatur urutan giliran (turn order) pemain dan musuh.
  - v. Sistem harus menampilkan waktu pertarungan yang dimiliki pemain untuk mengalahkan musuh. Jika pertarungan belum selesai dan waktu telah habis, maka player akan dinilai kalah dalam pertarungan.
  - vi. Sistem harus mengatur kondisi akhir pertarungan, baik kemenangan (menampilkan layar reward dan menyimpan progres) maupun kekalahan (memberikan opsi mengulang).
- c) *Enemy Behavior*
  - i. Logika Serangan Statis: NPC (musuh) harus dapat memberikan serangan kepada party pemain saat gilirannya tiba.
  - ii. Logika Penargetan Dinamis: AI harus dapat memilih target pemain secara dinamis

- (misal: menargetkan karakter dengan HP terendah).
- iii. Perilaku Berdasarkan Fase (HP Trigger):: AI musuh (terutama bos) harus dapat mengeluarkan skill khusus ketika HP-nya mencapai ambang batas tertentu (misalnya, di bawah 75%, 50%, atau 25%)..
  - iv. Perilaku Reaktif : AI harus dapat merespons aksi pemain saat charge bar penuh(misal: menggunakan *refresh* saat menerima banyak debuff).
  - v. Manajemen Serangan Spesial (*Charge Bar*): Musuh bos harus memiliki mekanisme charge bar untuk melepaskan serangan pamungkas yang tiap barnya terisi saat boss melancarkan serangan biasa atau saat menerima serangan.
  - vi. Penanda Serangan Kuat (*Telegraphing*): AI harus memberikan sinyal/peringatan satu giliran sebelum menggunakan serangan yang sangat kuat.
  - vii. Implementasi Mekanisme (*Rage Bar*): Sistem akan mengimplementasikan 'Rage Bar' untuk musuh bos yang terisi saat menerima damage.
  - viii. Implementasi "*Enraged Phase*": Saat *Rage Bar* penuh, AI bos akan masuk ke fase '*Enraged*' yang mengubah prioritas dan variasi skill-nya.
  - ix. Implementasi Fase "*Exhausted Phase*": Setelah '*Enraged*' berakhir, bos akan menerima

debuff signifikan (fase kelelahan) untuk beberapa turn.

- x. Indikator UI untuk *Rage Bar*: Sistem harus menyediakan indikator visual yang jelas untuk status *Rage Bar* pada antarmuka (glow outline pada HP bar).

d) *Sistem Gacha*

- i. Pemain dapat melakukan Pull gacha dengan currency dalam game.
- ii. Logika probabilitas untuk menentukan hasil Gacha harus dieksekusi sepenuhnya di sisi server (backend) untuk menjamin keamanan dan keadilan.
- iii. Sistem harus mengimplementasikan "Pity System": Jika pemain tidak mendapatkan item dengan raritas SSR setelah sejumlah tarikan yang ditentukan (misal: 39 tarikan), maka tarikan berikutnya dijamin akan menghasilkan item SSR. Counter ini akan di-reset setelah pemain mendapatkan SSR.
- iv. Sistem harus secara otomatis menambahkan hasil Gacha (karakter/senjata) ke dalam inventaris pemain.

e) *Quest & Progression*

- i. Sistem harus menyediakan antarmuka visual (peta atau daftar) bagi pemain untuk melihat dan memilih stage/quest yang tersedia.
- ii. Pemain dapat menyelesaikan quest dan menerima reward.
- iii. Sistem harus mencatat dan menyimpan progress quest pemain ke dalam database.

- iv. Sistem harus mengimplementasikan mekanisme Energi/Stamina yang dibutuhkan untuk memulai quest dan beregenerasi seiring waktu.
- f) Manajemen *Party*
  - i. Pemain dapat menambahkan, menghapus, dan mengatur anggota party.
  - ii. Pemain dapat menambahkan, menghapus, dan mengatur grid senjata party.
  - iii. Pemain dapat merubah skill yang digunakan karakter utama untuk dipakai dalam pertarungan.
  - iv. Sistem harus memungkinkan pemain untuk menyimpan beberapa konfigurasi tim yang berbeda (party preset).
- g) Manajemen *Inventory*
  - i. Sistem harus menyimpan dan menampilkan item dan karakter yang dimiliki pemain.
  - ii. Sistem harus menampilkan detail item saat pemain memilih sebuah item atau karakter.
  - iii. Pemain dapat menyortir item atau karakter dalam inventory.
  - iv. Pemain dapat meningkatkan level (enhance) senjata atau karakter dengan menggunakan material enhancement tertentu.
  - v. Pemain dapat meningkatkan batas level maksimal (limit break) pada karakter yang dipilih dengan material spesifik yang diperlukan.
  - vi. Pemain dapat memilih satu atau lebih untuk membongkar (dismantle) atau menjual (sell) item dalam inventory yang tidak diinginkan.

Sistem harus memberikan konfirmasi dan informasi item atau sumber daya apa saja yang didapat pemain sebelum proses dieksekusi.

h) Pengaturan dan Logout

- i. Sistem harus menyediakan menu di mana pemain dapat mengatur opsi game (misalnya, volume suara, kualitas grafis).
- ii. Sistem harus menyediakan opsi bagi pemain untuk keluar (logout) dari akun mereka secara aman.

2. Kebutuhan Non-Fungsional

1. Kinerja

- a. Waktu Respons UI (Klien): Interaksi di antarmuka pengguna (seperti menekan tombol menu, membuka inventaris) harus memberikan respons visual dalam  $< 200$  milidetik.
- b. Waktu Respons Server: Permintaan data ke server (seperti memuat data pemain atau memulai pertarungan) harus selesai dalam  $< 1000$  milidetik (1 detik) dalam kondisi jaringan normal.
- c. Pertarungan harus berjalan dengan frame stabil minimal 30 fps.

2. Keamanan

- a. Data akun pemain harus disimpan dengan enkripsi.
- b. Setiap aksi kritis (penggunaan currency, Gacha, penjualan item) harus divalidasi di sisi server.

3. Reliabilitas

Sistem harus tetap berjalan dengan kondisi 10+ pemain online secara bersamaan.

4. Portabilitas

Game dapat dijalankan pada browser modern (Chrome, Firefox, Edge).

5. Usability

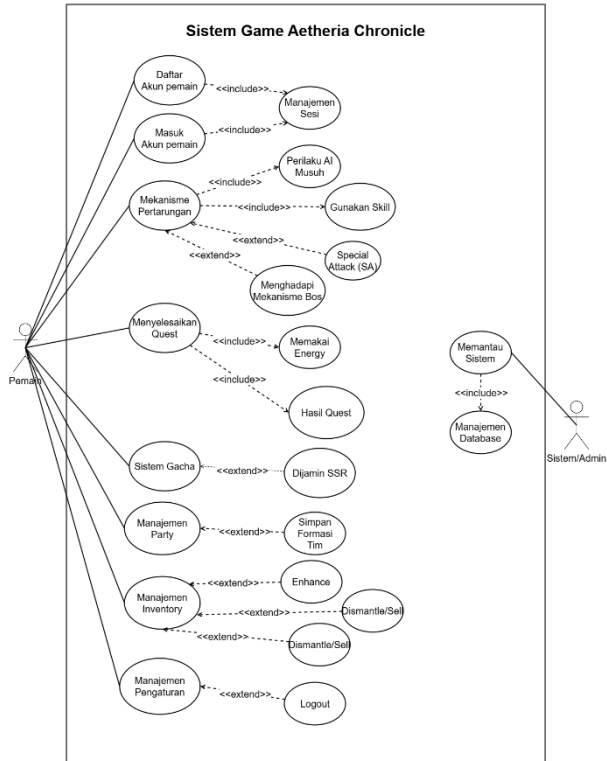
- a. Konsistensi Desain Antarmuka: Komponen UI seperti tombol, ikon, dan menu harus memiliki desain dan perilaku yang konsisten di seluruh aplikasi.
- b. Feedback Visual untuk Aksi Pengguna: Sistem harus memberikan umpan balik visual (misalnya, tombol berubah warna saat ditekan, animasi memuat) untuk setiap aksi yang dilakukan pemain.
- c. Antarmuka game akan ditampilkan dengan rasio aspek potret (9:16). Tampilan ini akan diposisikan di tengah layar dan ukurannya akan menyesuaikan dengan tinggi jendela browser tanpa mengubah rasio aspeknya. Area kosong di sisi kanan dan kiri akan diisi dengan latar belakang yang sesuai.

### 3.1.2 Desain Sistem

Tahap desain sistem merupakan fase Design dalam metodologi XP. Pada fase ini, dilakukan pemodelan arsitektur sistem untuk memastikan rancangan perangkat lunak mudah dipahami, diubah, dan dikembangkan oleh pengembang.

Proses ini mencakup pembuatan use case diagram, activity diagram, class diagram, *Entity Relationship Diagram (ERD)*, serta rancangan arsitektur AI yang menjadi inti kecerdasan musuh dalam game. Setiap hasil desain pada tahap ini akan divalidasi melalui peer review dan umpan balik pengguna sesuai prinsip XP, sebelum dilanjutkan ke tahap implementasi untuk memastikan kesesuaian antara rancangan dan kebutuhan sistem yang telah dianalisis.

#### a. Use Case Model



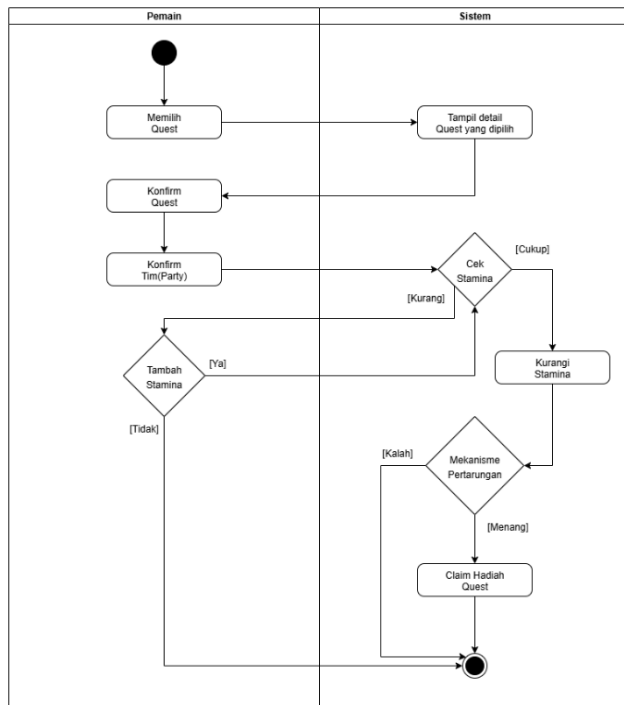
Gambar 3.2 Diagram *Usecase*

Diagram *Use Case* pada Gambar 3.2 ini memodelkan interaksi tingkat tinggi antara aktor dengan fungsionalitas utama dari "Sistem Game Aetheria Chronicle".

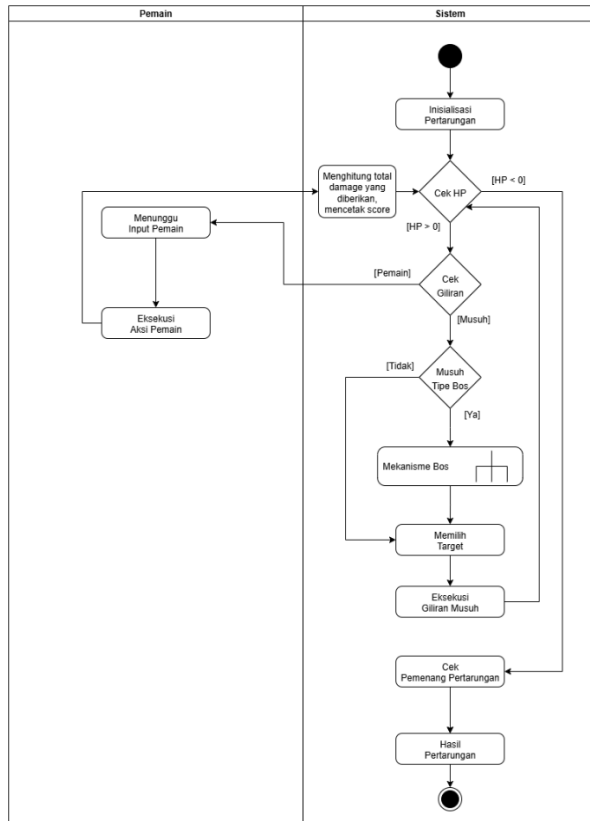
Terdapat dua aktor utama, yaitu Pemain dan Sistem/Admin. Aktor Pemain memiliki akses ke delapan *use case* inti, yang mencakup seluruh alur permainan mulai dari Daftar Akun pemain dan Masuk Akun pemain, Menyelesaikan Quest, Mekanisme Pertarungan, Sistem Gacha, Manajemen Party, Manajemen Inventory, hingga Manajemen Pengaturan.

Diagram juga memetakan memetakan relasi <<include>> untuk fungsionalitas yang wajib ada (seperti Perilaku AI Musuh yang merupakan bagian dari Mekanisme Pertarungan) dan relasi <<extend>> untuk fungsionalitas opsional atau situasional (seperti Menghadapi Mekanisme Bos sebagai perluasan dari pertarungan biasa, atau Dijamin SSR sebagai perluasan dari Sistem Gacha). Selain itu, terdapat aktor Sistem/Admin yang memiliki tugas untuk Memantau Sistem, yang mencakup Manajemen Database.

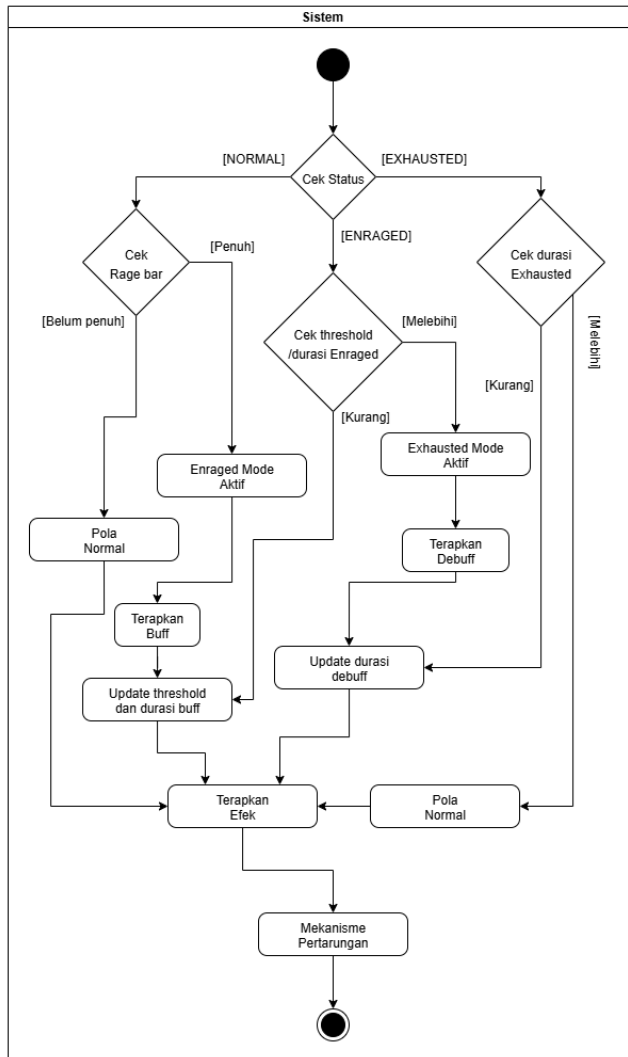
b. Activity Diagram



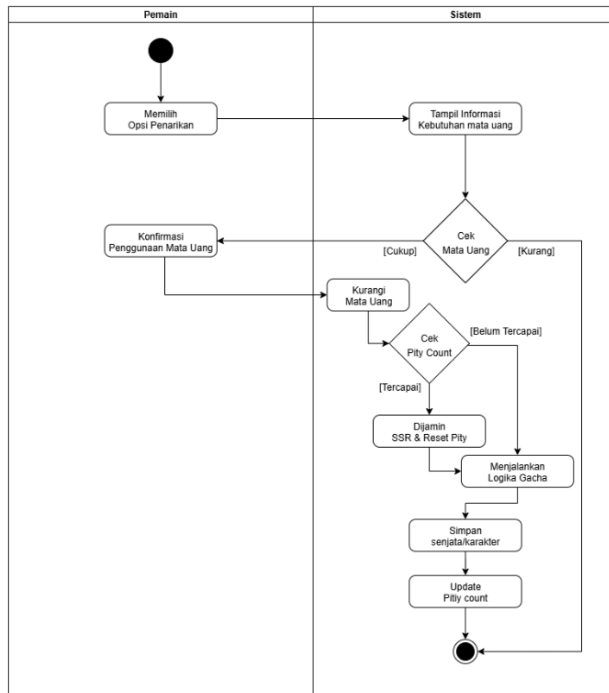
Gambar 3.3 Aktivitas Quest



Gambar 3.4 Aktivitas Pertarungan



Gambar 3.5 Sub-aktivitas Mekanisme Bos

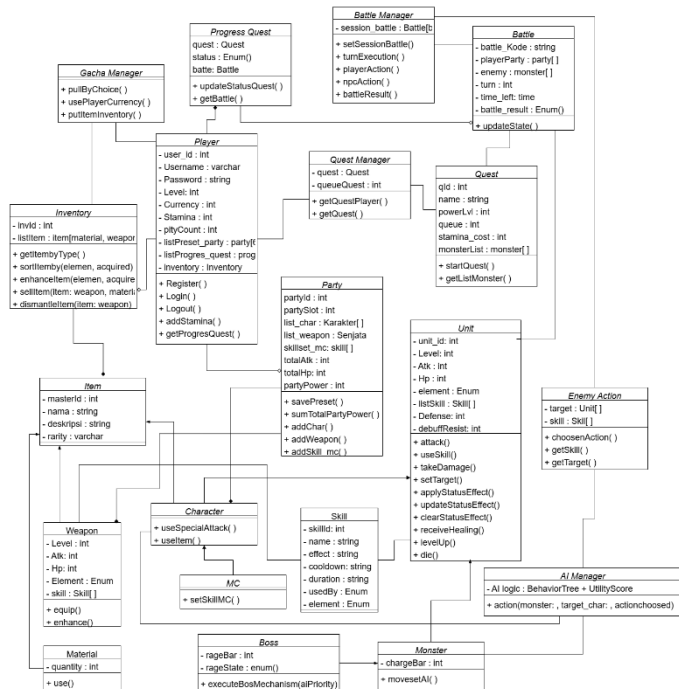


Gambar 3.6 Aktivitas Gacha

Secara keseluruhan, alur kerja utama sistem Aetheria Chronicle dimodelkan melalui serangkaian diagram Aktivitas yang saling terhubung untuk merepresentasikan proses utama dalam permainan.

1. Diagram Aktivitas Quest menggambarkan alur pemain sejak pemilihan misi, pengecekan ketersediaan stamina, hingga proses konfirmasi tim dan pemanggilan mekanisme pertarungan. Diagram ini juga menunjukkan kondisi percabangan ketika stamina tidak mencukupi atau ketika pemain berhasil menyelesaikan misi dan mengklaim hadiah.

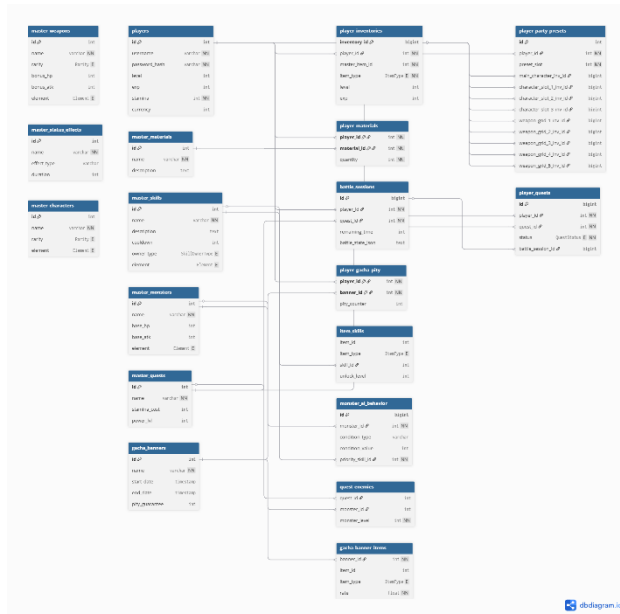
2. Diagram Aktifitas Pertarungan memodelkan *core gameplay loop* berbasis *turn-based combat*. Diagram ini mencakup proses pemeriksaan status HP, pergantian giliran, eksekusi aksi pemain atau musuh, hingga penentuan hasil akhir pertarungan.
  3. Untuk musuh bertipe bos, alur dilanjutkan pada Sub-Diagram Aktifitas Mekanisme Bos, yang secara khusus memvisualisasikan perubahan status AI bos antara fase *Normal*, *Enraged*, dan *Exhausted*. Transisi ini menggambarkan logika perilaku dinamis yang dipengaruhi oleh kondisi seperti *Rage Bar* dan durasi efek *buff/debuff* yang aktif selama pertarungan.
  4. Diagram Aktifitas Gacha menjelaskan proses perolehan karakter atau senjata melalui sistem gacha, dimulai dari pemilihan opsi penarikan, validasi mata uang dalam permainan, hingga penerapan *pity system* yang menjamin perolehan item langka (*SSR*). Hasil gacha yang diperoleh kemudian disimpan secara otomatis ke dalam inventaris pemain.
- c. Class Diagram



Gambar 3.7 Class Diagram

Diagram Kelas pada Gambar 3.7 berfungsi sebagai cetak biru untuk struktur kode aplikasi. Diagram ini mendefinisikan kelas-kelas utama, beserta atribut (data) dan operasinya (metode/perilaku). Selain itu, diagram ini juga memetakan hubungan antar kelas seperti pewarisan (*inheritance*), komposisi, dan asosiasi, yang menjadi fondasi dalam implementasi sistem berorientasi objek.

d. ERD

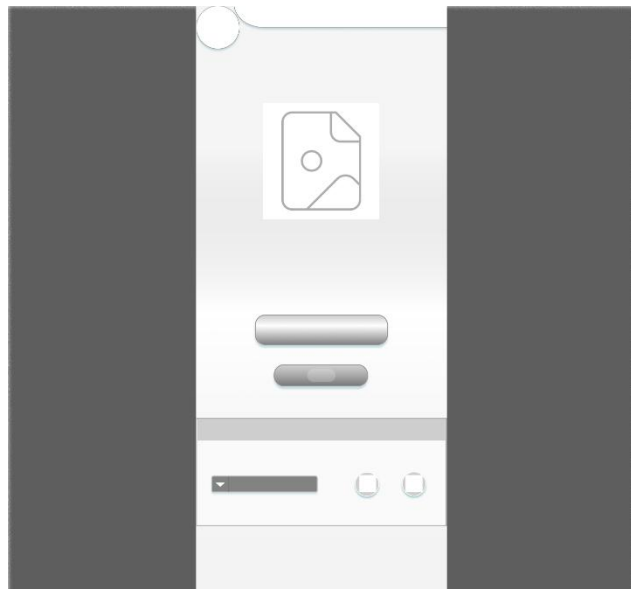


Gambar 3.8 ERD

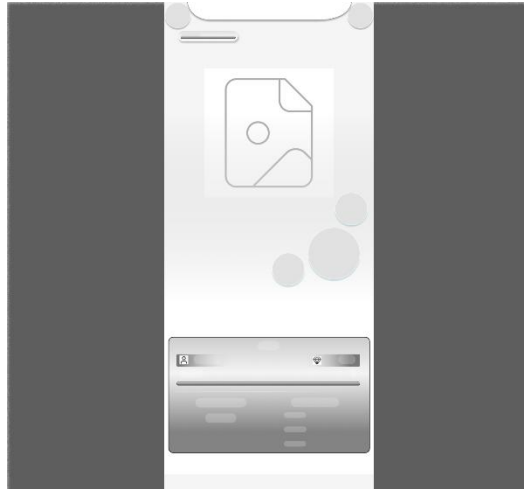
Diagram ERD pada Gambar 3.8 berfungsi sebagai cetak biru untuk struktur logis dari *database* MySQL yang akan digunakan. Rancangan ini memisahkan secara tegas antara Tabel Master yang berisi definisi konten statis game (seperti *master\_characters* dan *master quests*) dengan Tabel Transaksi yang melacak data dinamis dan unik untuk setiap pemain (seperti *player\_inventories* dan *player quests*). Melalui penggunaan *Primary Key*, *Foreign Key*, dan tabel jembatan seperti *item\_skills*, skema ini membentuk relasi *one-to-many* dan *many-to-many* yang memastikan integritas data serta mendukung semua fungsionalitas inti, termasuk persistensi sesi pertarungan, penyimpanan *party preset*, dan progres *quest* individual.

e. Desain Wireframe

Sub-bab berikut menyajikan rancangan antarmuka pengguna (UI/UX) dalam bentuk *wireframe low-fidelity*. *Wireframe* ini bertujuan untuk memvisualisasikan tata letak (layout), alur navigasi, dan penempatan komponen-komponen fungsional utama pada layar-layar paling krusial dalam permainan "Aetheria Chronicles", mulai dari layar judul, layar utama, hingga layar pertarungan.



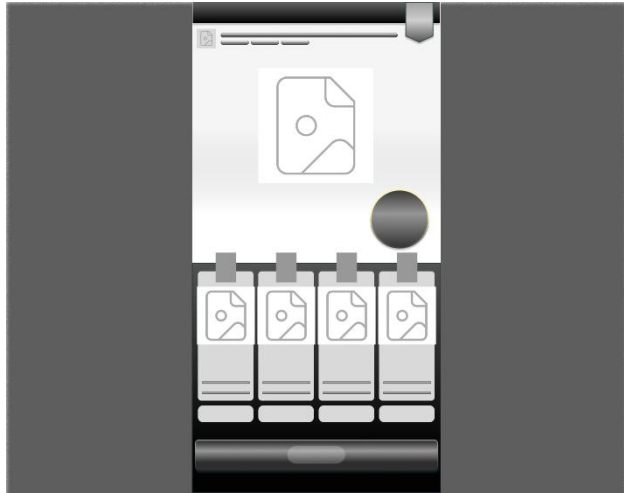
Gambar 3.9 *Title Screen*



Gambar 3.10 *Home Screen*



Gambar 3.11 *Quest Selection*



Gambar 3.12 *Battle Screen*



Gambar 3.13 *Gacha Screen*

### 3.1.3 Implementasi Sistem

Fase ini merepresentasikan tahap Coding dalam siklus *Extreme Programming (XP)*. Pada tahap ini, setiap modul yang telah dirancang mulai diimplementasikan melalui proses pengkodean berulang (*incremental coding*) menggunakan pendekatan *pair programming* dan *continuous integration* untuk menjaga konsistensi kualitas kode. Implementasi sistem dilakukan berdasarkan hasil perancangan yang telah dibuat pada tahap sebelumnya, meliputi penerapan *Entity Relationship Diagram (ERD)*, *Class Diagram*, *Activity Diagram*, serta Rancangan Arsitektur AI yang dikembangkan. Proses implementasi dilakukan menggunakan framework Phaser.js sebagai mesin utama pengembangan game berbasis web. Phaser.js dipilih karena mendukung pengelolaan *scene*, *sprite*, animasi, serta sistem pertarungan *turn-based* secara efisien di sisi klien.

Pada sisi backend, sistem menggunakan Node.js dengan Express.js sebagai server application untuk menangani logika bisnis dan komunikasi dengan database MySQL. API berbasis REST digunakan untuk mengatur komunikasi antara frontend dan backend dalam pengambilan serta penyimpanan data pemain seperti status quest, data pertarungan, inventory, party configuration, dan hasil gacha. Autentikasi pengguna diterapkan menggunakan JWT (JSON Web Token) untuk menjaga keamanan session pemain.

Setiap modul dihubungkan melalui manager classes seperti *BattleManager*, *QuestManager*, *GachaManager*, dan *AIManager* yang bertugas mengatur alur logika permainan dan koordinasi antar komponen. Antarmuka pengguna dibangun berdasarkan desain wireframe yang telah disusun sebelumnya, mencakup tampilan Title Screen, Home Screen, Quest Selection,

Battle Scene, Gacha Interface, Inventory Screen, dan Party Management.

Database MySQL dirancang sesuai dengan ERD yang telah dibuat, dengan normalisasi hingga bentuk normal ketiga (3NF) untuk memastikan integritas data dan menghindari redundansi. Implementasi dilakukan secara iteratif mengikuti prinsip *Extreme Programming* (XP), di mana setiap modul dikembangkan, diuji, dan diintegrasikan secara bertahap untuk memastikan fungsionalitas yang stabil sebelum melanjutkan ke modul berikutnya.

### **3.1.4 Pengujian & Evaluasi Sistem**

Tahap Pengujian dan Evaluasi Sistem merupakan bagian dari fase Testing dalam metode *Extreme Programming* (XP), di mana sistem yang telah diimplementasikan diuji secara menyeluruh untuk memastikan kualitas dan fungsionalitasnya. Tahap pengujian sistem bertujuan untuk memvalidasi bahwa prototipe game *Aetheria Chronicles* telah memenuhi semua kebutuhan fungsional dan non-fungsional yang telah didefinisikan. Proses ini dilakukan secara sistematis untuk memastikan kualitas, kinerja, dan stabilitas aplikasi sebelum evaluasi akhir.

#### **a. Metode Pengujian**

Pendekatan pengujian yang digunakan dalam penelitian ini adalah sebagai berikut:

1. Pengujian Fungsional (Black Box Testing) Metode pengujian utama yang digunakan adalah Black Box Testing. Pengujian ini berfokus pada verifikasi fungsionalitas sistem dari sudut pandang pengguna, di mana setiap fitur diuji berdasarkan input yang diberikan dan output yang diharapkan, tanpa melihat struktur kode internal. Metode ini dipilih karena sangat relevan untuk memastikan bahwa setiap *use*

*case* yang telah dirancang berjalan sesuai dengan skenarionya.

2. Pengujian Non-Fungsional Pengujian ini dilakukan untuk mengukur aspek kualitas sistem di luar fungsionalitas inti, seperti kinerja, kompatibilitas antar peramban, dan usabilitas antarmuka.
- b. Skenario Pengujian

Skenario pengujian dikembangkan berdasarkan kebutuhan fungsional yang telah dianalisis sebelumnya. Setiap skenario akan didokumentasikan dalam sebuah tabel pengujian untuk mencatat hasil secara sistematis. Ruang lingkup pengujian mencakup, namun tidak terbatas pada, skenario-skenario kunci berikut:

1. Akun & Login: Memvalidasi proses registrasi, login dengan kredensial yang benar dan salah, serta fungsionalitas logout.
2. Mekanisme Pertarungan: Menguji akurasi perhitungan damage multi-layer, eksekusi skill dan cooldown, penerapan efek buff/debuff, serta penentuan kondisi menang/kalah.
3. AI Musuh: Menguji logika pengambilan keputusan AI pada berbagai kondisi, termasuk eksekusi mekanisme bos (*Enraged* dan *Exhausted*).
4. Sistem Gacha: Memvalidasi mekanisme penarikan, validasi currency, dan fungsionalitas pity system.
5. Sistem Quest & Persistensi: Menguji alur penyelesaian quest, distribusi reward, dan kemampuan sistem untuk melanjutkan pertarungan yang tertunda setelah login kembali.
6. Manajemen Inventaris & Tim: Memvalidasi fungsi tambah/hapus item, serta penyimpanan dan pemanggilan party preset.

c. Kriteria Evaluasi Sistem

Setelah tahap pengujian selesai, evaluasi sistem akan dilakukan berdasarkan kriteria berikut untuk mengukur keberhasilan proyek:

1. Verifikasi Fungsional: Hasil dari semua skenario black box testing akan direkapitulasi untuk menentukan persentase keberhasilan implementasi fitur dibandingkan dengan daftar kebutuhan fungsional.
2. Analisis Kinerja: Data dari pengujian non-fungsional (seperti FPS rata-rata dan waktu respons) akan dianalisis dan dibandingkan dengan target yang telah ditetapkan (misalnya, minimal 30 FPS).
3. Identifikasi Bug dan Limitasi: Semua bug atau anomali yang ditemukan selama pengujian akan dicatat, diklasifikasikan berdasarkan tingkat keparahannya (*severity*), dan didokumentasikan sebagai potensi perbaikan atau limitasi dari sistem yang dibangun.

Hasil evaluasi ini akan menjadi landasan untuk menarik kesimpulan penelitian dan memberikan rekomendasi untuk pengembangan lebih lanjut, sejalan dengan siklus umpan balik pada metodologi *Extreme Programming* (XP).

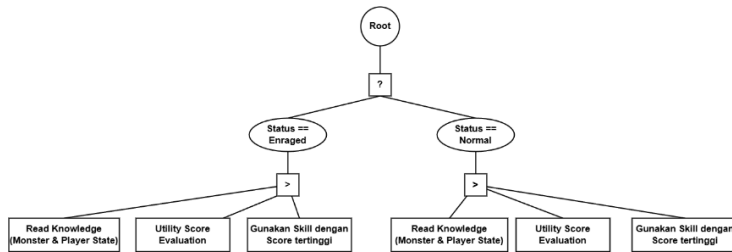
### 3.2 Arsitektur AI

Arsitektur AI Adaptif akan dirancang untuk melengkapi bagian dari game yang kerap menggunakan AI *scripted* yang polanya mudah ditebak. Dengan menggabungkan dua algoritma, Behavior Tree (BT) sebagai pengatur logika, dan Utility Score sebagai pengambil keputusan.

Arsitektur ini bertujuan untuk membangun perilaku NPC yang dinamis meskipun strukturnya statis. Hal ini dicapai dengan

membuat AI mampu mengevaluasi berbagai data (*Knowledge*) secara *real-time* untuk memilih aksi yang paling optimal.

### 3.2.1 Struktur Behavior Tree



Gambar 3.14 Diagram Behavior Tree

Dari Gambar 3.14, struktur BT dimulai dari root node yang mengarah ke selector node guna menentukan status kondisi musuh (*Normal* atau *Enraged*). Masing-masing status memiliki sequence node yang memproses langkah-langkah pengambilan keputusan secara berurutan, dimulai dari pembacaan kondisi pertempuran (*Read Knowledge*), evaluasi nilai utilitas (*Utility Score Evaluation*), hingga eksekusi aksi (*Execute Action*).

Selector berfungsi memilih jalur logika yang aktif sesuai kondisi terkini, sedangkan sequence memastikan bahwa proses pengambilan data dan evaluasi berjalan secara sistematis sebelum aksi dijalankan. Dengan begitu, BT memastikan AI tetap modular, terstruktur, dan mudah dikembangkan untuk berbagai tipe musuh di masa depan.

### 3.2.2 Basis Pengetahuan (*Knowledge Base*)

Pada bagian Node (*Read Knowledge*) dalam BT berfungsi untuk mengumpulkan informasi situasional secara *real-time* dalam pertarungan. Data yang dikumpulkan dibagi dalam dua kategori:

- a. Internal (Monster)

1. HP Level, klasifikasi kondisi kesehatan musuh yang mempengaruhi Tingkat agresifitas.
  2. Tipe Monster & Skill set, masing-masing monster memiliki tipe dan kombinasi skill yang berbeda. Misalnya, monster A memiliki Single Target High Damage, sementara monster B memiliki AoE Attack dan Heal/Refresh.
  3. Status Monster, klasifikasi tingkat Kesehatan monster dengan banyaknya jumlah debuff yang aktif yang mempengaruhi perilaku Defensif.
- b. Eksternal (Party Player)
1. HP Terendah Party, menentukan target potensial jika AI memiliki kecenderungan menyerang karakter yang lemah.
  2. ATK Tertinggi di Party, target prioritas bagi tipe AI agresif untuk mengurangi output damage pemain.
  3. Jumlah Buff dan Debuff Party, menjadi acuan untuk skill bertipe Cleanse, Dispel, atau Amplify Damage.
  4. Special Attack Readiness (SA Bar), menentukan ancaman party dan memengaruhi kemungkinan AI menggunakan skill bertipe defensif.

Data di atas digunakan AI untuk menyusun keputusan secara dinamis di setiap giliran. Jadi meskipun struktur BT bersifat statis, tetapi Keputusan yang diambil bisa berbeda – beda tergantung kondisi pertarungan yang berlangsung.

### **3.2.3 Model Keputusan (*Utility Score*)**

Tahap Utility Score Evaluation merupakan inti dari pengambilan keputusan adaptif AI. Setelah semua data dari Read Knowledge dikumpulkan, sistem akan memberikan nilai utilitas (*utility score*) untuk setiap skill yang dimiliki monster berdasarkan kondisi saat itu.

Setiap skill memiliki bobot parameter yang berbeda tergantung pada fungsinya – misalnya, skill bertipe ofensif akan lebih sensitif terhadap HP pemain dan jumlah buff aktif, sedangkan skill bertipe defensif akan lebih memperhatikan HP monster dan jumlah debuff yang diterima.

Tabel 3.1 Tabel Utility Score (Normal)

| Skill                                | Value | M_<br>Status<br>(>2) | P_HP<br>(High) | P_Buff<br>(> 2) | Final |
|--------------------------------------|-------|----------------------|----------------|-----------------|-------|
| <i>Dark Slash</i><br>(Single Target) | 1.0   | - 0.2                | + 0.5          | + 0.3           | 1.8   |
| <i>Abyss Roar</i><br>(AoE + Dispel)  | 0.8   | - 0.4                | + 0.5          | + 0.8           | 1.7   |
| <i>Shadow Mend</i><br>(Heal/Refresh) | 0.5   | + 0.9                | - 0.2          | - 0.1           | 1.1   |

Tabel 3.2 Tabel Utility Score (Enraged)

| Skill                                   | Value | M_<br>Status<br>(>2) | P_HP<br>(High) | P_Buff<br>(> 2) | Final |
|-----------------------------------------|-------|----------------------|----------------|-----------------|-------|
| <i>Dark Slash +</i><br>(Single Target)  | 1.5   | - 0.2                | + 0.5          | + 0.3           | 2.1   |
| <i>Abyss Roar</i><br>+(AoE +<br>Dispel) | 1.3   | - 0.4                | + 0.5          | + 0.8           | 2.2   |
| <i>Shadow Mend</i><br>(Heal/Refresh)    | 0.3   | + 0.9                | - 0.2          | - 0.1           | 1.1   |

AI kemudian memilih skill dengan nilai utilitas tertinggi untuk dieksekusi pada giliran tersebut. Proses ini membuat AI terlihat “berpikir” dan menyesuaikan taktiknya dengan kondisi permainan, tanpa mengubah struktur pohon keputusan yang ada.

Dengan mengombinasikan Behavior Tree sebagai *decision framework* dan Utility Score sebagai *adaptive evaluation system*, AI musuh dalam *Aetheria Chronicles* mampu menampilkan perilaku

yang dinamis, namun tetap dapat dikontrol. Struktur BT memastikan alur logika tetap teratur dan mudah dikembangkan, sedangkan Utility Score memastikan setiap aksi yang diambil kontekstual dan sesuai dengan situasi pertempuran.

## DAFTAR PUSTAKA

- [1] Newzoo, "Global Games Market Report 2024," Aug. 2024.
- [2] Sensor Tower and Dataai, "State of Mobile Gaming 2024 Market milestones, emerging trends, and exclusive insights," 2024.
- [3] V. Chergarova, M. Tomeo, H. Morgan, and S. Andrade, "Integrating artificial intelligence (AI) into game development to elevate diverse gameplay elements," *Issues in Information Systems*, vol. 25, no. 2, pp. 408–417, 2024, doi: 10.48009/2\_iis\_2024\_132.
- [4] Game Developer Conference and Game Developer, "2024 State of The Game Industry," 2024.
- [5] Y. A. Sekhavat, "Behavior Trees for Computer Games," 2017.
- [6] M. Iovino, E. Scukins, J. Styrud, P. Ögren, and C. Smith, "A survey of Behavior Trees in robotics and AI," Aug. 01, 2022, *Elsevier B.V.* doi: 10.1016/j.robot.2022.104096.
- [7] A. Sánchez-Ruiz, S. Lee-Urban, H. Muñoz, M. Muñoz-Avila, B. Díaz-Agudo, and P. González-Calero, "Game AI for a Turn-based Strategy Game with Plan Adaptation and Ontology-based retrieval," 2007. [Online]. Available: [www.aaai.org](http://www.aaai.org)
- [8] J. Schell, "The Art of Game Design: A Book of Lenses," 2008.
- [9] D. Turangan, "PEMBANGUNAN GAME RPG TURN-BASED MENGGUNAKAN ALGORITMA BRUTE FORCE," 2020.

- [10] R. Marcotte and H. J. Hamilton, "Behavior Trees for Modelling Artificial Intelligence in Games: A Tutorial," *The Computer Games Journal*, vol. 6, no. 3, pp. 171–184, Sep. 2017, doi: 10.1007/s40869-017-0040-9.
- [11] M. Attar Zoghi and S. John Sharar, "Evolving Monetization: A Formal Analysis of Gacha Mechanics in Online Multiplayer Games from 2005 to 2025 Faculty of Arts Department of Game Design," 2025.
- [12] Andriyanto, "EXPLORATORY STUDY OF HOW GACHA SYSTEM IN MOBILE VIDEO GAME AFFECT PLAYER'S ENJOYMENT: A CASE STUDY OF INDONESIA," 2023.
- [13] D. Stiawan, C. Prabowo, A. Heryanto, N. Afifah, A. E. Minarno, and R. Budiarto, "Visualization of Granblue Fantasy Game Traffic Pattern Using Deep Packet Inspection Method," *Kinetik: Game Technology, Information System, Computer Network, Computing, Electronics, and Control*, pp. 169–178, Jul. 2020, doi: 10.22219/kinetik.v5i3.1073.
- [14] Cygames.co, "Granblue Fantasy." Accessed: Sep. 28, 2025. [Online]. Available: <https://granbluefantasy.jp/>
- [15] V. Mäkelä and A. Schmidt, "I Don't Care as Long as It's Good: Player Preferences for Real-Time and Turn-Based Combat Systems in Computer RPGs," in *CHI PLAY 2020 - Proceedings of the Annual Symposium on Computer-Human Interaction in Play*, Association for Computing Machinery, Inc, Nov. 2020, pp. 231–240. doi: 10.1145/3410404.3414248.
- [16] J. M. Domingues, V. Filipe, A. Carita, and V. Carvalho, "Understanding the Impact of Perceived Challenge on Narrative Immersion in Video Games: The Role-Playing

- Game Genre as a Case Study," *Information (Switzerland)*, vol. 15, no. 6, Jun. 2024, doi: 10.3390/info15060294.
- [17] F. Amereh, "A Study and Implementation of Turn-Based Combat Systems in Role-Playing Games," 2024.
  - [18] TTRPG Directory, "Ultimate Guide to Balancing Turn-Based Combat." Accessed: Sep. 28, 2025. [Online]. Available: <https://www.ttrpg-games.com/blog/ultimate-guide-to-balancing-turn-based-combat/>
  - [19] Y. Su, P. Backlund, and H. Engström, "Comprehensive review and classification of game analytics," in *Service Oriented Computing and Applications*, Springer Science and Business Media Deutschland GmbH, Jun. 2021, pp. 141–156. doi: 10.1007/s11761-020-00303-z.
  - [20] M. Colledanchise and P. Ögren, "Behavior Trees in Robotics and AI: An Introduction," Oct. 2022, doi: 10.1201/9780429489105.
  - [21] V. Rudramalla *et al.*, "A Comprehensive Review of AI Techniques in Serious Games: Decision Making and Machine Learning," *International Journal for Modern Trends in Science and Technology*, vol. 10, no. 02, pp. 305–311, 2024, doi: 10.46501/IJMTST1002039.
  - [22] J. McGuire, "AI Decision-Making with Utility Scores (Part 1) - Forty Years of Code." Accessed: Sep. 28, 2025. [Online]. Available: <https://mcguirev10.com/2019/01/03/ai-decision-making-with-utility-scores-part-1.html>
  - [23] S. Mbuguah, G. Nyabuto, and V. Mony, "Client-server Architecture, a Review," *International Journal of Advanced*

*Science and Computer Applications*, vol. 3, no. 1, Dec. 2023, doi: 10.47679/ijasca.v3i1.48.

- [24] Phaser, "What is Phaser?" Accessed: Sep. 28, 2025. [Online]. Available: <https://docs.phaser.io/phaser/getting-started/what-is-phaser>
- [25] Travis Faas, "An Introduction To HTML5 Game Development with Phaser.js," 2017.
- [26] Brown, "Web Development with Node and Express," 2019.
- [27] T. Taipalus, "Database management system performance comparisons: A systematic literature review," *Journal of Systems and Software*, vol. 208, Feb. 2024, doi: 10.1016/j.jss.2023.111872.
- [28] V. B. Ramu, "Optimizing Database Performance: Strategies for Efficient Query Execution and Resource Utilization," *International Journal of Computer Trends and Technology*, vol. 71, no. 7, pp. 15–21, Jul. 2023, doi: 10.14445/22312803/ijctt-v71i7p103.
- [29] A. Junaidi, A. Yunus, and S. A. Wiguna, "IMPLEMENTASI BEHAVIOR TREE PADA PERILAKU NPC DI GAME SIDESCROLLER," *KURAWAL Jurnal Teknolgi, Informasi dan Industri*, 2021.
- [30] B. P. Indah and S. Handayani, "FACTORS THAT INFLUENCE GACHA GAME USERS' DECISION-MAKING IN USING GACHA GAMES," *DIPONEGORO JOURNAL OF ACCOUNTING*, vol. 13, no. 4, pp. 1–15, 2024, [Online]. Available: <http://ejournal-s1.undip.ac.id/index.php/accounting>

- [31] R. Marcotte and H. J. Hamilton, "Behavior Trees for Modelling Artificial Intelligence in Games: A Tutorial," *The Computer Games Journal*, vol. 6, no. 3, pp. 171–184, Sep. 2017, doi: 10.1007/s40869-017-0040-9.
- [32] A. Solihin and Y. Yamasari, "Pengembangan Game Ular Tangga Aritmatika Berbasis Web," *Journal of Informatics and Computer Science*, vol. 05, 2023.
- [33] Y. Bimatara Putra, E. M. Adams Jonemaro, and M. A. Akbar, "Penerapan Adaptive AI pada Game Turn Based RPG Dengan Menggunakan Metode Monte Carlo Tree Search," 2018. [Online]. Available: <http://j-ptiik.ub.ac.id>
- [34] R. Karsten, J. Pragantha, and D. A. Haris, "Perancangan Game Turn Based 'Elemental Fight' pada Platform HTML5," 2024.
- [35] Sugiyono, "METODE PENELITIAN KUANTITATIF, KUALITATIF, DAN RND," 2019.
- [36] Dovetail, "What is Extreme Programming? An Overview of XP." Accessed: Sep. 28, 2025. [Online]. Available: <https://dovetail.com/product-development/what-is-extreme-programming/>
- [37] M. Iwan Wahyuddin, R. Tamara Aldisa, and I. Diana Sholihati, "Sistem Informasi Administrasi Kemahasiswaan dan Alumni (Smart Adma) dengan Metode Extreme Programming (XP)," *Jurnal Teknologi Informasi dan Komunikasi*, vol. 5, no. 4, p. 2021, 2021, doi: 10.35870/jti.
- [38] A. Sasongko, B. Lailiah, P. S. Hadikusumah, and Y. Marien, "Implementasi Agile Pengembangan Sistem Informasi

Karyawan Berbasis Next.js dan PostgreSQL: Studi Kasus Yayasan Almadani," 2025.